

Laboratoire 3 — Affichage de l'heure sur LCD avec TIM2

Matisse Rhéaume-Viale

6 octobre 2026

Cours : 243-356-LI — Programmation 3 — Microcontrôleurs et périphériques

Enseignant : Gabriel Fortin-Bélanger

Groupe : 0001

1. Objectif

Le but est d'afficher l'heure au format HH:MM:SS sur l'écran LCD. Le Timer 2 déclenche une interruption chaque seconde. L'interruption avance l'heure et la boucle principale met l'écran à jour.

Le programme utilise les registres du STM32F103 directement. Il n'utilise pas HAL. L'écran est piloté en mode 4 bits avec les broches DB4 à DB7.

2. Branchement de l'écran

Le plan du laboratoire omet la broche E (Enable). Le pilote LCD utilisé au Lab 1 confirme que E est reliée à PB11. Il faut donc aussi brancher cette broche.

Signal LCD	Broche STM32F103	Rôle
RS	PB9	Sélection commande ou donnée
R/W	PB10	Sélection lecture ou écriture
E	PB11	Validation du transfert
DB4	PB12	Donnée, bit 4
DB5	PB13	Donnée, bit 5
DB6	PB14	Donnée, bit 6
DB7	PB15	Donnée, bit 7



INTERFACE PIN FUNCTIONS

Pin No.	Symbol	Level	Description
1	VSS	0V	Ground.
2	VDD	+5.0V	Power supply for logic operating.
3	V0	--	Adjusting supply voltage for LCD driving.
4	RS	H/L	A signal for selecting registers: 1: Data Register (for read and write) 0: Instruction Register (for write), Busy flag-Address Counter (for read).
5	R/W	H/L	R/W = "H": Read mode. R/W = "L": Write mode.
6	E	H/L	An enable signal for writing or reading data.
7	DB0	H/L	This is an 8-bit bi-directional data bus.
8	DB1	H/L	
9	DB2	H/L	
10	DB3	H/L	
11	DB4	H/L	
12	DB5	H/L	
13	DB6	H/L	
14	DB7	H/L	
15	LED+	+5.0V	Power supply for backlight.
16	LED-	0V	The backlight ground.

Figure 1 — Brochage du module TC2004A. Tinsarp, p. 5.

Le code écrit seulement dans l'écran. Il garde donc R/W à 0. La fonction LCD_Commande() ne fait rien si RW vaut true, car son prototype ne permet pas de retourner la valeur lue et les lignes de données sont configurées en sorties.

À confirmer avant le branchement : la fiche du module TC2004A indique une alimentation logique de 5 V et un niveau haut minimal qui dépend de cette alimentation. Il faut vérifier la fiche du module exact et la compatibilité entre ses entrées et les sorties 3,3 V du STM32. Le câblage électrique n'a pas été vérifié sur la carte pendant ce travail.

3. Fonctionnement du programme

Au démarrage, `SystemInit()` sélectionne le HSI à 8 MHz et laisse les préscaleurs AHB et APB à 1. `LCD_Init()` configure les broches et démarre l'écran en mode 4 bits. Le programme affiche ensuite 00:00:00, puis démarre TIM2.

TIM2 compte une seconde par période. Son interruption met à jour le nombre de secondes depuis minuit et envoie l'événement **SEV**. La boucle principale attend avec **WFE**. Au réveil, elle vérifie si le compteur de mise à jour a changé, puis rafraîchit l'écran. Les écritures LCD restent donc dans le programme principal et ne ralentissent pas l'interruption.

Le temps est gardé sous forme d'un nombre de secondes entre 0 et 86 399. Après 23:59:59, il revient à zéro. Cette représentation rend le passage à la minute et à minuit simple à gérer.

4. Plan de test

Les essais ci-dessous viennent des consignes du Lab 3. Aucun résultat Saleae n'est présenté comme mesuré : les essais sur carte et les captures de l'analyseur logique restent à faire.

Cas	Vérification	Résultat attendu	Résultat obtenu
1	Capturer une commande LCD avec l'analyseur logique	Pour 0x28, RS=0, R/W=0, puis les nibbles 0010 et 1000, chacun validé par une impulsion sur E.	NON TESTÉ — capture matérielle absente.
2	Mesurer la précision de l'affichage des secondes	Le début de chaque rafraîchissement de l'heure est espacé d'environ 1 s. Noter les périodes minimale, moyenne et maximale mesurées.	NON TESTÉ — capture matérielle absente.
3	Vérifier le retour à zéro	En partant de 23:59:55, l'écran doit afficher 00:00:00 après cinq interruptions d'une seconde.	NON TESTÉ — capture matérielle absente.

4.1 Procédures proposées

Cas 1 — Transmission d'une commande. Brancher les voies de l'analyseur sur RS, R/W, E et DB4 à DB7, puis redémarrer la carte. Capturer l'initialisation et repérer la commande 0x28. Vérifier RS=0, R/W=0, les deux nibbles dans le bon ordre et une impulsion E par nibble.

Cas 2 — Intervalle des secondes. Capturer plusieurs rafraîchissements consécutifs sur E. Pour chaque rafraîchissement de l'heure, noter l'instant du premier front montant de E. Calculer les intervalles entre ces instants et rapporter le minimum, la moyenne et le maximum. La valeur attendue est proche de 1 s ; la précision réelle dépend notamment de l'horloge de la carte.

Cas 3 — Passage de minuit. Pour cet essai seulement, remplacer l'initialisation à zéro dans `main()` par :

```
(void)Timer2_SetTime(23U, 59U, 55U);
```

Après la compilation et le redémarrage, vérifier la suite 23:59:55, 23:59:56, 23:59:57, 23:59:58, 23:59:59, puis 00:00:00. Remettre ensuite l'initialisation normale à zéro. Cette procédure n'a pas été exécutée sur le matériel.

5. Analyse des fonctions demandées

5.1 LCD_Init()

Extrait du code source

```
void LCD_Init(void)
{
    LCD_GPIO_Init();

    /* SysTick sert ici de compteur libre pour les délais du LCD. */
    SYST_CSR = 0UL;
    SYST_RVR = SYST_RELOAD;
    SYST_CVR = 0UL;
    SYST_CSR = SYST_CLKSOURCE | SYST_ENABLE;

    LCD_Delai_Ms(50UL);

    LCD_Envoyer_Nibble(0x03U);
    LCD_Delai_Us(4500UL);
    LCD_Envoyer_Nibble(0x03U);
    LCD_Delai_Ms(1UL);
    LCD_Envoyer_Nibble(0x03U);
    LCD_Delai_Us(150UL);
    LCD_Envoyer_Nibble(0x02U);
    LCD_Delai_Us(150UL);

    LCD_Commande(false, false, LCD_CMD_FUNCTION_SET);
    LCD_Commande(false, false, LCD_CMD_DISPLAY_OFF);
    LCD_Commande(false, false, LCD_CMD_CLEAR);
    LCD_Commande(false, false, LCD_CMD_ENTRY_MODE);
    LCD_Commande(false, false, LCD_CMD_DISPLAY_ON);
}
```

Explication

LCD_GPIO_Init() active l'horloge de GPIOB, configure PB9 à PB15 comme sorties push-pull et met les sorties à zéro. SysTick est démarré sans interruption; il sert de compteur libre pour mesurer les délais. À 8 MHz, chaque microseconde correspond à 8 cycles.

Après 50 ms, le code envoie les nibbles 0011, 0011, 0011, puis 0010. Cette séquence place le contrôleur LCD en mode 4 bits. Les commandes suivantes configurent l'écran, effacent son contenu et activent l'affichage. 0x28 choisit le transfert 4 bits et le format de caractère 5 × 8 points.

Registre	Adresse	Utilisation dans le pilote
RCC_APB2ENR	0x40021018	Bit 3 à 1 pour activer GPIOB.
GPIOB_CRH	0x40010C04	Configure PB9 à PB15 en sorties push-pull à 2 MHz (MODE=10, CNF=00).
GPIOB_BSRR	0x40010C10	Met les broches de commande et de données à 0 sans opération de lecture-modification-écriture.
SysTick_CSR	0xE000E010	Sélectionne l'horloge processeur et démarre le compteur; TICKINT reste à 0.
SysTick_RVR	0xE000E014	Fixe le rechargement à la valeur maximale de 24 bits.

Registre	Adresse	Utilisation dans le pilote
SysTick_CVR	0xE000E018	Fournit la valeur descendante utilisée pour calculer les délais.

Les extraits du manuel STM32 ci-dessous montrent le bit d'activation de GPIOB, la configuration de GPIOx_CRH et le registre de sortie GPIOx_BSRR.

RM0008 Low-, medium-, high- and XL-density reset and clock control (RCC)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										TIM11 EN	TIM10 EN	TIM9 EN	Reserved		
										rw	rw	rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC3 EN	USART 1EN	TIM8 EN	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	IOPG EN	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.	AFIO EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw

Bits 31:22 Reserved, must be kept at reset value.

Bit 21 **TIM11EN**: TIM11 timer clock enable

Set and cleared by software.

0: TIM11 timer clock disabled

1: TIM11 timer clock enabled

Bit 20 **TIM10EN**: TIM10 timer clock enable

Set and cleared by software.

0: TIM10 timer clock disabled

1: TIM10 timer clock enabled

Figure 2 — Registre de configuration et activation de GPIOB. ST RM0008, p. 113.

General-purpose and alternate-function I/Os (GPIOs and AFIOs)

RM0008

9.2.2 Port configuration register high (GPIOx_CRH) (x=A..G)

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]	MODE15[1:0]	CNF14[1:0]	MODE14[1:0]	CNF13[1:0]	MODE13[1:0]	CNF12[1:0]	MODE12[1:0]	CNF11[1:0]	MODE11[1:0]	CNF10[1:0]	MODE10[1:0]	CNF9[1:0]	MODE9[1:0]	CNF8[1:0]	MODE8[1:0]
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]	MODE11[1:0]	CNF10[1:0]	MODE10[1:0]	CNF9[1:0]	MODE9[1:0]	CNF8[1:0]	MODE8[1:0]	CNF7[1:0]	MODE7[1:0]	CNF6[1:0]	MODE6[1:0]	CNF5[1:0]	MODE5[1:0]	CNF4[1:0]	MODE4[1:0]
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, **CNFy[1:0]**: Port x configuration bits (y= 8 .. 15)

23:22, 19:18, 15:14, These bits are written by software to configure the corresponding I/O port.

11:10, 7:6, 3:2 Refer to [Table 20: Port bit configuration table](#).

In input mode (MODE[1:0]=00):

00: Analog mode

01: Floating input (reset state)

10: Input with pull-up / pull-down

11: Reserved

In output mode (MODE[1:0] > 00):

00: General purpose output push-pull

01: General purpose output Open-drain

10: Alternate function output Push-pull

11: Alternate function output Open-drain

Bits 29:28, 25:24, **MODEy[1:0]**: Port x mode bits (y= 8 .. 15)

21:20, 17:16, 13:12, These bits are written by software to configure the corresponding I/O port.

9:8, 5:4, 1:0 Refer to [Table 20: Port bit configuration table](#).

00: Input mode (reset state)

01: Output mode, max speed 10 MHz.

10: Output mode, max speed 2 MHz.

11: Output mode, max speed 50 MHz.

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Figure 3 — Configuration des broches PB8 à PB15. ST RM0008, p. 172.

9.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y= 0 .. 15)

These bits can be read and written by software and can be accessed in Word mode only.

Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Bit set/reset bit (y= 0 .. 15)

Figure 4 — Registre de mise à 1 et de remise à 0. ST RM0008, p. 173.

La fiche HD44780U montre le transfert en deux nibbles ainsi que les délais d'initialisation. Le schéma de séquence ci-dessous vient de la documentation du cours utilisée au Lab 1.

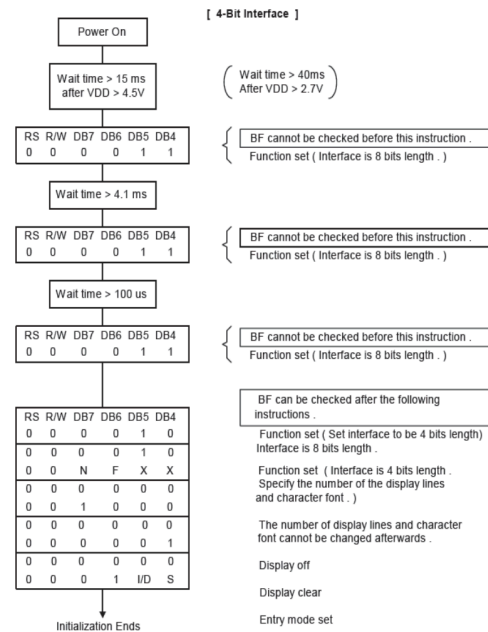


Figure 5 — Séquence de démarrage en mode 4 bits. Documentation du cours, Lab 1.

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3). The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

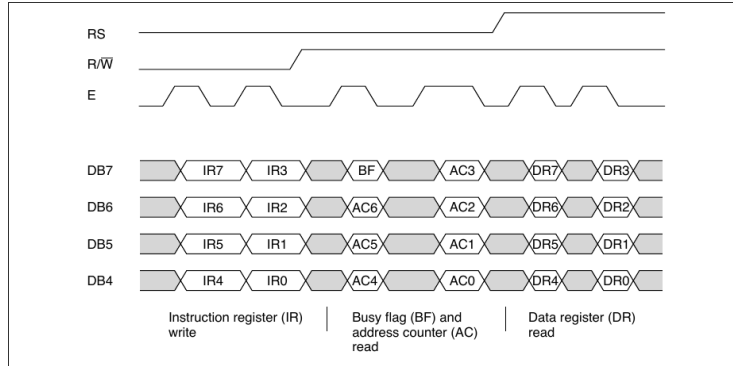


Figure 9 4-Bit Transfer Example

Figure 6 — Transfert et timing de l'interface 4 bits. Hitachi HD44780U, p. 22.

5.2 LCD_Commande()

Extrait du code source

```
void LCD_Commande(bool RW, bool RS, uint8_t Data)
{
    if (RW)
    {
        return;
    }

    LCD_SetPin(LCD_RS_PIN, RS);
    LCD_SetPin(LCD_RW_PIN, false);

    LCD_Envoyer_Nibble((uint8_t)(Data >> 4U));
    LCD_Envoyer_Nibble((uint8_t)(Data & LCD_DATA_MASK));

    if ((!RS) && ((Data == LCD_CMD_CLEAR) || (Data == LCD_CMD_HOME)))
    {
        LCD_Delai_Ms(2UL);
    }
    else
    {
        LCD_Delai_Us(50UL);
    }
}
```

Explication

RS=0 sélectionne une commande; RS=1 sélectionne un caractère. Le pilote écrit toujours avec R/W=0. L'octet est séparé en deux groupes de quatre bits : le nibble de poids fort est transmis en premier, puis celui de poids faible. LCD_Envoyer_Nibble() place les quatre bits sur PB12 à PB15 et produit une impulsion sur PB11.

Le registre GPIOB_BSRR met chaque broche à 1 ou à 0. Pour éviter de toucher aux autres sorties, le code écrit les bits de mise à 1 ou de remise à 0 dans une seule écriture au registre. Les commandes CLEAR et HOME ont un délai de 2 ms; les autres opérations attendent 50 µs.

La figure du fabricant confirme l'ordre des nibbles dans une interface 4 bits. Pour 0x28, le bus présente d'abord 0010, puis 1000.

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3). The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

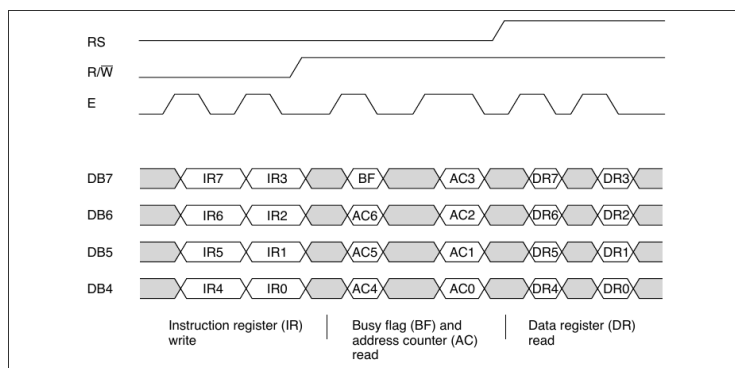


Figure 9 4-Bit Transfer Example

Figure 7 — Ordre des transferts en mode 4 bits. Hitachi HD44780U, p. 22.

5.3 LCD_Ecrire_Char()

Extrait du code source

```
void LCD_Ecrire_Char(unsigned char caractere, uint8_t ligne, uint8_t colonne)
{
    uint8_t adresse;

    if ((ligne < 1U) || (ligne > LCD_ROWS)
        || (colonne < 1U) || (colonne > LCD_COLUMNS))
    {
        return;
    }

    adresse = (uint8_t)(LCD_CMD_SET_DDRAM
                        | adresseLigne[ligne - 1U]
                        | (uint8_t)(colonne - 1U));
    LCD_Commande(false, false, adresse);
    LCD_Commande(false, true, (uint8_t)caractere);
}
```

Explication

Les numéros de ligne et de colonne commencent à 1. La fonction vérifie qu'ils sont dans les limites d'un écran 20×4 . Elle ajoute le décalage de la colonne à l'adresse de base de la ligne. Les adresses utilisées sont 0x00, 0x40, 0x14 et 0x54.

Le bit 7 de l'instruction est mis à 1 pour choisir une adresse DDRAM. La fonction envoie d'abord cette instruction avec RS=0, puis le caractère avec RS=1. Le tableau d'adresses correspond à la disposition mémoire du contrôleur pour un écran 20×4 .

		HD44780U
Set DDRAM Address		
Set DDRAM address sets the DDRAM address binary AAAAAA into the address counter.		
Data is then written to or read from the MPU for DDRAM.		
However, when N is 0 (1-line display), AAAAAA can be 00H to 4FH. When N is 1 (2-line display), AAAAAA can be 00H to 27H for the first line, and 40H to 67H for the second line.		
Read Busy Flag and Address		
Read busy flag and address reads the busy flag (BF) indicating that the system is now internally operating on a previously received instruction. If BF is 1, the internal operation is in progress. The next instruction will not be accepted until BF is reset to 0. Check the BF status before the next write operation. At the same time, the value of the address counter in binary AAAAAA is read out. This address counter is used by both CG and DDRAM addresses, and its value is determined by the previous instruction. The address contents are the same as for instructions set CGRAM address and set DDRAM address.		
Table 7 Shift Function		
S/C	R/L	
0	0	Shifts the cursor position to the left. (AC is decremented by one.)

Figure 8 — Instruction et adresses DDRAM. Hitachi HD44780U, p. 29.

5.4 LCD_Affichage_Temps()

Extrait du code source

```
void LCD_Affichage_Temps(void)
{
    uint32_t temps;
    uint8_t heures;
    uint8_t minutes;
    uint8_t secondes;
    unsigned char caracteres[8];
    uint8_t colonne;

    temps = Timer2_GetSecondsSinceMidnight();
    heures = (uint8_t)(temps / 3600UL);
    minutes = (uint8_t)((temps / 60UL) % 60UL);
    secondes = (uint8_t)(temps % 60UL);

    caracteres[0] = (unsigned char)('0' + (heures / 10U));
    caracteres[1] = (unsigned char)('0' + (heures % 10U));
    caracteres[2] = ':';
    caracteres[3] = (unsigned char)('0' + (minutes / 10U));
    caracteres[4] = (unsigned char)('0' + (minutes % 10U));
    caracteres[5] = ':';
    caracteres[6] = (unsigned char)('0' + (secondes / 10U));
    caracteres[7] = (unsigned char)('0' + (secondes % 10U));

    for (colonne = 0U; colonne < 8U; colonne++)
    {
        LCD_Ecrire_Char(caracteres[colonne], 1U, (uint8_t)(colonne + 1U));
    }
}
```

Explication

La fonction lit le compteur de secondes, puis calcule les heures, les minutes et les secondes. Chaque chiffre est converti en caractère ASCII en ajoutant sa valeur à '0'. Elle envoie ensuite les huit caractères sur la première ligne, de la colonne 1 à la colonne 8.

Cette fonction ne configure pas de registre matériel elle-même. Les transferts se font par `LCD_Ecrire_Char()` et `LCD_Commande()`. La figure du fabricant ci-dessous montre comment les caractères passent au LCD par son interface 4 bits.

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3). The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

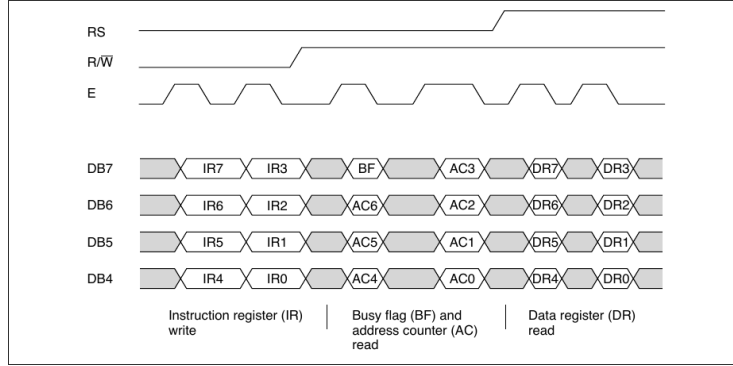


Figure 9 4-Bit Transfer Example

Figure 9 — Transfert d'un octet vers l'écran en mode 4 bits. Hitachi HD44780U, p. 22.

6. Réglage de TIM2

Le HSI est à 8 MHz. Comme le préscalier APB1 vaut 1, l'horloge de TIM2 est aussi à 8 MHz. Le code utilise PSC=7999 et ARR=999 :

$$f_{\text{update}} = \frac{8\,000\,000}{(7999 + 1)(999 + 1)} = 1 \text{ Hz}$$

TIM2 produit donc une interruption chaque seconde. Le registre TIM2_EGR reçoit UG pour appliquer le préscalier. Le code efface ensuite les indicateurs en attente, active UIE dans TIM2_DIER, puis active l'IRQ 28 dans le NVIC.

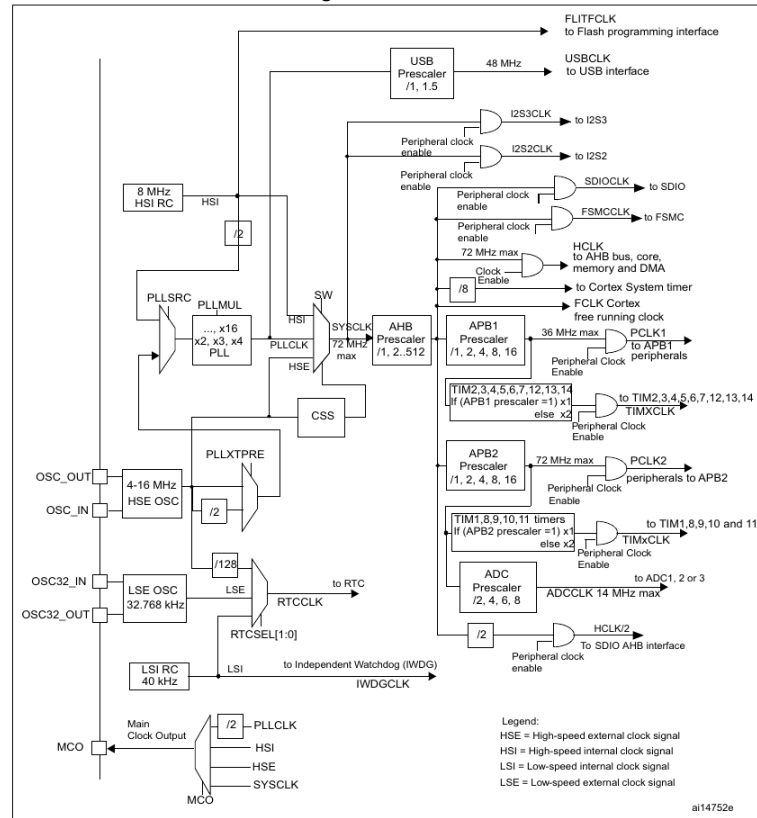
Registre	Adresse	Valeur ou rôle
RCC_APB1ENR	0x4002101C	Bit 0 à 1 pour activer TIM2.
TIM2_CR1	0x40000000	URS choisit les demandes Update du compteur ; CEN démarre le timer.
TIM2_DIER	0x4000000C	UIE autorise l'interruption Update.
TIM2_SR	0x40000010	L'indicateur UIF signale une période terminée.
TIM2_EGR	0x40000014	UG applique immédiatement les valeurs de base du timer.
TIM2_PSC	0x40000028	Divise l'horloge par PSC + 1 = 8000.
TIM2_ARR	0x4000002C	Compte ARR + 1 = 1000 impulsions avant Update.

Registre	Adresse	Valeur ou rôle
NVIC_ISER0	0xE000E100	Bit 28 à 1 pour activer l'IRQ TIM2.

RM0008

Low-, medium-, high- and XL-density reset and clock control (RCC)

Figure 8. Clock tree



1. When the HSI is used as a PLL clock input, the maximum system clock frequency that can be achieved is 64 MHz.

Figure 10 — Arbre d'horloge STM32F103. ST RM0008, p. 93.

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value
The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in "reset mode").

15.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **ARR[15:0]**: Prescaler value
ARR is the value to be loaded in the actual auto-reload register.
Refer to the [Section 15.3.1: Time-base unit](#) for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

Address offset: 0x34

Reset value: 0x0000

Figure 11 — Définition des préscaler et auto-reload de TIM2. ST RM0008, p. 419.

7. Vérification logicielle et limites

La compilation et l'édition de liens croisées du firmware avec `arm-none-eabi-gcc` ont réussi. L'image obtenue contient 2 004 octets de code et 1 576 octets de BSS. L'éditeur de liens a affiché des avertissements de stubs système newlib (`_close`, `_read`, `_write`, `_lseek`) ; il s'agit des appels système non fournis pour cette application bare metal. Aucun résultat de compilation ne prouve le fonctionnement sur une carte.

Les trois essais avec l'analyseur logique, l'affichage réel du LCD, la précision du HSI et le passage physique à minuit restent **NON TESTÉS**. Il faudra ajouter les captures de l'analyseur et les observations de la carte après les essais.

Le dossier `build/qc1` présent dans Lab 3 a été copié du Lab 2 et son cache CMake référence toujours les sources du Lab 2. Il n'a pas été utilisé pour la compilation ci-dessus. Le firmware a été compilé dans un répertoire temporaire séparé afin de préserver ces fichiers existants.

8. Conclusion

Le code du Lab 3 configure le LCD en mode 4 bits, place les caractères à l'adresse DDRAM voulue et convertit le compteur de secondes en affichage HH:MM:SS. TIM2 est configuré pour une interruption d'une seconde et le passage de 23:59:59 à 00:00:00 est prévu dans le code. La compilation croisée a réussi. Il reste à vérifier le branchement électrique, exécuter les trois essais sur la carte et ajouter leurs captures réelles.

Références

- Consignes du laboratoire : `TEP_Prog_Lab3.pdf`, fourni avec le travail.
- Plan du laboratoire : `Laboratoires/Lab_3/plan.md`.

- Documentation du contrôleur : Hitachi, *HD44780U Dot Matrix Liquid Crystal Display Controller/Driver*, fichier ../../../../Documentation/Datashit/HD44780.pdf, pages 22 et 29.
- Documentation du module : Tinsarp, *TC2004A*, fichier ../../../../Documentation/Datashit/TC2004A-01.pdf, page 5.
- Documentation STM32 : STMicroelectronics, *RM0008 Reference Manual*, fichier RM0008 dans ../../../../Documentation/Datashit/; pages 93, 113, 172–173 et 418–419.
- Rapport du Lab 1 : ../../../../Documentation/RapportLab1.pdf ; utilisé pour confirmer le branchement de E sur PB11.
- Rapport du Lab 2 : Laboratoires/Lab_2/RapportLab2.md ; utilisé comme référence pour la structure et la présentation du rapport.